Box 1. Summary of Best Practices

1. Write programs for people, not computers.

(a) 一个程序不应要求其读者持有更多
而不是记忆中的少数事实
(b) 使名称一致、独特和有意义。 (c) 使代码样式和格
式一致。

2. 由计算机进行工作。

(a) 使计算机重复工作。
(b) 将最近的命令保存在文件中供重新使用。
(c) 使用构建工具实现工作流程自动化。

3. 进行渐进式改革。

(a) 采取小步骤,并经常得到反馈和课程。
更正。
(b) 使用版本控制系统。 (c) 将所有手工创建的软件都放入
版本中。
控制系
统。

4. 不要重复自己(或其他人)。

(a) 每一件数据必须具有单一的权威。
系统中的代表性。
(b) 修改代码,而不是复制和粘贴。
(c) 重新使用代码,而不是重写代码。

5. 错误计划。

(a) 在程序上添加断言,以检查其运行情况。
(b) 使用现成单元测试库。
(c) 将虫子变成试验病例。
(d) 使用象征性调试器。

6. 只有在软件正常工作后才能优化。

(a) 使用剖面仪查明瓶颈。
(b) 用尽可能高的语言写代码。

7. 文件设计和目的,而不是机械。

(a) 文件接口和理由,而不是执行。
(b) 重构代码,而不是解释其运作方式。
(c) 将一件软件的文件嵌入其中;
软件。

8. 合作。

(a) 使用合并前的代码审查。
(b) 在使新人跟上速度和处理特别棘手问题时,
使用对等编程。
(c) 使用问题跟踪工具。

为人写程序,

不是计算机 科学家写软件需要写出两者都执行的代码
正确和容易被他人阅读和理解
程序员(特别是作者的未来自我). 如果软件
阅读和理解不易,
知道它实际上正在做它要做的事。
因此,软件开发者必须从几个方面着手
人类认识、特别是
工作内存有限,人类模式匹配能力是
精细的调谐,人类的注意力跨度很短[33,37].
首先,一个程序不应该要求其读者持有
一时记忆中不止几个事实(1a)。
人类的工作记忆只能保存在
时间,每个项目要么是一个单一的事实,要么是一个"圆"
合计数项事实[33,34],所以程序应限制总数
为完成某项任务而需要记住的项目数量。
实现这个目标的主要方式是 将程序简单化
理解函数,每个函数都进行单个,容易
理解,任务。这可以使每个程序
更容易以同样的方式理解 打破科学
使用部分和段落的纸张使阅读更加容易。
第二,科学家应使名称一致、 独特和有意义(1b)。

名字,像 a 和 o, 或者名字非常相似, 像结果和
结果 2 可能会造成混乱。
第三,科学家应制作代码样式和格式
如果使用的科学论文的不同部分
报告的格式和资本化方式不同,因此,文件
同样,如果程序的不同部分是
缩进不同, 或者程序员混合骆驼 CaseNaming
和 坑洞案件 命名,代码需要更长的阅读和阅读时间
制造更多的错误[35,36]。

让“计算机

工作科学”经常涉及重复计算任务,例如
以相同方式处理大量数据文件或
每次将新数据添加到现有数据中时,都会重新生成数字。
计算机被发明来做这种重复
任务,但即使是今天,许多科学家也会在
反复或反复单击相同的按钮[17]。在
除了浪费时间 迟早甚至是最小心的
研究人员在这样做时会失去注意力,犯错误。
因此,科学家应该让计算机重复任务
(2a),并将最近的命令保存在一个文件中以供重新使用(2b).
例如,大多数命令行工具都有“历史”选项
允许用户显示并重新执行最近的命令,使用
次要编辑为文件名或参数。这经常被引用为一个
原因 命令线接口 仍为流行 [38,39]:"这样做
用于节省时间和减少错误。
包含交互式系统命令的文件往往
虽然这和
程序。当这些脚本在同一程序中反复使用时
工作流程管理工具可以是
正在汇编和链接
堡垒、C++、爪哇语和 C#等语言的程序
[40] (中文(简体)).这个任务最广泛使用的工具可能是 制作
(http://www..gnu.org/software/make),
现在有电动车[41]。
文件之间的依赖关系,即如果 A 或 B 有变化,
然后需要使用一组特定的指令更新 C 。
这些工具已成功用于科学工作。
[42] (中文(简体)).
避免重复命令出错和效率低下
我们建议科学家使用一个构建工具
自动工作流程(2c),例如,指定如何
中间数据文件和最终结果相互依赖,
用于创建它们的程序,以便一个单一的命令
重新生成任何需要重新生成的东西。